SEV 세션 관리

개요

AMD SEV 세션 관리는 테넌트(게스트 소유자)와 AMD 보안 프로세서(SP) 간의 보안 채널을 설정하고 관리하는 핵심 프로세스입니다. 이 문서는 보안 세션 설정 프로세스, 키 교환 및 통신 보안 메커니즘, 세션 암호화 및 데이터 보호 방법을 상세히 설명합니다.

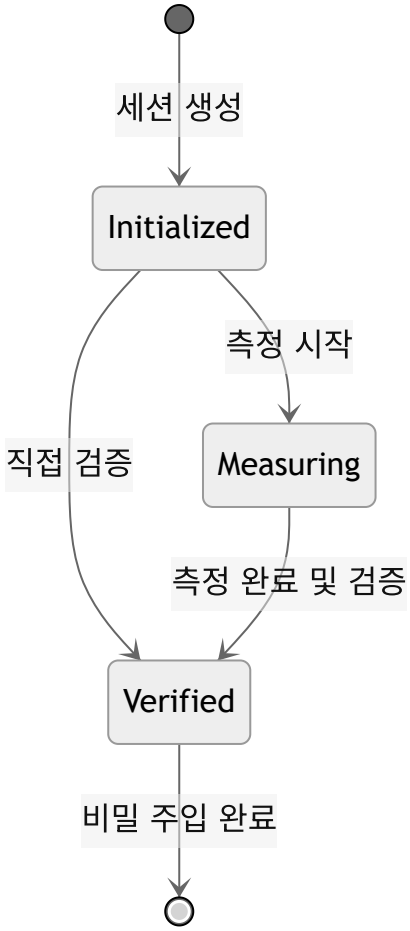
세션 관리 개요

1. 세션의 목적

SEV 세션은 다음과 같은 목적을 가집니다:

- **보안 통신**: 테넌트와 AMD SP 간의 암호화된 통신 채널 제공
- **무결성 보장**: 전송되는 데이터의 무결성 검증
- **인증**: 통신 당사자의 신원 확인
- **비밀 주입**: 게스트에 안전하게 비밀 정보 전달

2. 세션 상태 머신



세션 초기화

1. 세션 생성

```
use sev::session::Session;

use sev::launch::sev::{Policy, PolicyFlags};

use sev::firmware::host::Version;


// 보안 정책 설정

let  policy = Policy {

flags: PolicyFlags::NO_DEBUG | PolicyFlags::ENCRYPTED_STATE,

minfw: Version::default(),
```

```
};

// 세션 초기화
let session = Session::try_from(policy)?;
```

2. 키 생성

세션 초기화 시 두 개의 핵심 키가 생성됩니다:

```
use sev::session::key::Key;

// Transport Encryption Key (TEK) - 16바이트
let tek = Key::random(16)?;

// Transport Integrity Key (TIK) - 16바이트
let tik = Key::random(16)?;
```

3. 키 보안 특성

```
// 키는 메모리에서 안전하게 관리됩니다
impl Drop for Key {
fn drop(&mut self) {
// 메모리에서 키 데이터를 안전하게 제거
for b in self.0.iter_mut() {
```

```
unsafe {
write_volatile(b  as *mut u8, 0u8);
}
}
}
}
```

보안 채널 설정

1. Diffie-Hellman 키 교환

```
use sev::certs::sev::{Chain, sev::Certificate, sev::Usage};

// 인증서 체인 검증
let  chain = Chain::new(/* 인증서 체인 데이터 */)?;
let  pdh = chain.verify()?; // Platform Diffie-Hellman 인증서

// 임시 DH 키 쌍 생성
let (tenant_cert, tenant_priv) = Certificate::generate(Usage::PDH)?;

// 공유 비밀 생성
let  shared_secret = tenant_priv.derive(&pdh)?;
```

```rust
let z = Key::new(shared_secret);
```

2. 세션 매개변수 생성

```rust
use rdrand::RdRand;

// 랜덤 nonce와 IV 생성
let mut rng = RdRand::new()?;
let mut nonce = [0u8; 16];
let mut iv = [0u8; 16];

rng.try_fill_bytes(&mut nonce)?;
rng.try_fill_bytes(&mut iv)?;
```

3. 마스터 키 유도

```rust
// NIST 800-108 KDF를 사용한 키 유도
let master = z.derive(16, &nonce, "sev-master-secret")?;
let kek = master.derive(16, &[], "sev-kek")?; // Key Encryption Key
let kik = master.derive(16, &[], "sev-kik")?; // Key Integrity Key
```

키 교환 및 통신 보안

1. 키 래핑 (Key Wrapping)

```rust
use openssl::symm::{Crypter, Cipher, Mode};


// TEK와 TIK를 KEK로 암호화
let mut crypter = Crypter::new(
Cipher::aes_128_ctr(),
Mode::Encrypt,
&kek,
Some(&iv),
)?;


let mut wrapped_keys = [0u8; 32];
let mut offset = 0;


// TEK 암호화
offset += crypter.update(&tek, &mut wrapped_keys[offset..])?;
// TIK 암호화
offset += crypter.update(&tik, &mut wrapped_keys[offset..])?;
offset += crypter.finalize(&mut wrapped_keys[offset..])?;


assert_eq!(offset, 32); // TEK(16) + TIK(16)
```

2. 무결성 보호

```
// 래핑된 키의 무결성 보호
let wrap_mac = kik.mac(&wrapped_keys)?;


// 정책의 무결성 보호
let policy_bytes = policy.bytes(); // 4바이트
let policy_mac = tik.mac(&policy_bytes)?;
```

3. 세션 구조 생성

```
use sev::launch::sev::Session as LaunchSession;


let launch_session = LaunchSession {
nonce,
wrap_tk: wrapped_keys,
wrap_iv: iv,
wrap_mac,
policy_mac,
};
```

세션 암호화 및 데이터 보호

1. 런치 시작 데이터 생성

```rust
use sev::launch::sev::Start;


// 런치 시작에 필요한 모든 데이터 패키징
let start_data = Start {
policy,
cert: tenant_cert, // 테넌트 DH 인증서
session: launch_session, // 보안 세션 정보
};


// 이 데이터가 AMD SP로 안전하게 전송됩니다
let launcher = launcher.start(start_data)?;
```

2. 측정 과정에서의 세션 사용

```rust
// 측정 상태로 전환
let measuring_session = session.measure()?;


// 측정 데이터 업데이트
measuring_session.update_data(&guest_data)?;
measuring_session.update_data(&kernel_data)?;
measuring_session.update_data(&vmsa_data)?;
```

3. 측정 검증

```rust
use sev::firmware::host::Build;
use sev::launch::sev::Measurement;

// AMD SP로부터 받은 측정값
let sp_measurement = launcher.measurement();

// 펌웨어 빌드 정보
let build = Build {
version: Version { major: 0x00, minor: 0x12 },
build: 0x0f,
};

// 측정값 검증
let verified_session = measuring_session.verify(build, sp_measurement)?;
```

4. 측정 검증 알고리즘

```rust
// 내부적으로 수행되는 검증 과정
fn verify_measurement(
tik: &Key,
policy: &Policy,
digest: &[u8],
build: &Build,
```

```
measurement: &Measurement,
) -> Result<bool> {
let key = PKey::hmac(tik)?;
let mut signer = Signer::new(MessageDigest::sha256(), &key)?;
// 검증 데이터 구성
signer.update(&[0x04u8])?; // 버전 태그
signer.update(&[build.version.major])?; // 메이저 버전
signer.update(&[build.version.minor])?; // 마이너 버전
signer.update(&[build.build])?; // 빌드 번호
signer.update(&policy.bytes())?; // 정책
signer.update(digest)?; // 측정 다이제스트
signer.update(&measurement.mnonce)?; // 측정 nonce
let computed_mac = signer.sign_to_vec()?;
Ok(computed_mac == measurement.measure)
}
```

비밀 주입

1. 비밀 패킷 생성

```
use sev::launch::sev::{HeaderFlags, Secret, Header};

// 검증된 세션에서 비밀 생성
```

```rust
let secret_data = b"my-secret-key";
let flags = HeaderFlags::default(); // 압축 없음


let secret_packet = verified_session.secret(flags, secret_data)?;
```

2. 비밀 암호화 과정

```rust
// 내부적으로 수행되는 비밀 암호화
fn create_secret(
tek: &Key,
tik: &Key,
flags: HeaderFlags,
data: &[u8],
measurement: &Measurement,
) -> Result<Secret> {
// 랜덤 IV 생성
let mut iv = [0u8; 16];
let mut rng = RdRand::new()?;
rng.try_fill_bytes(&mut iv)?;
// 데이터 암호화
let ciphertext = symm::encrypt(
Cipher::aes_128_ctr(),
tek,
```

```
Some(&iv),
data
)?;
// MAC 계산
let key = PKey::hmac(tik)?;
let mut signer = Signer::new(MessageDigest::sha256(), &key)?;
signer.update(&[0x01u8])?; // 비밀 태그
signer.update(&flags.bits().to_le_bytes())?; // 플래그
signer.update(&iv)?; // IV
signer.update(&(data.len() as u32).to_le_bytes())?; // 원본 길이
signer.update(&(ciphertext.len() as u32).to_le_bytes())?; // 암호문 길이
signer.update(&ciphertext)?; // 암호문
signer.update(&measurement.measure)?; // 측정값
let mut mac = [0u8; 32];
signer.sign(&mut mac)?;
Ok(Secret {
header: Header { flags, iv, mac },
ciphertext,
})
}
```

3. 비밀 주입 실행

```
// 게스트 메모리 주소 지정
let guest_address = 0x1000;

// 비밀 주입
launcher.inject(&secret_packet, guest_address)?;
```

고급 세션 관리

1. 외부 다이제스트를 사용한 검증

```
// 외부에서 계산된 다이제스트 사용
let external_digest = calculate_external_digest(&guest_config)?;

let verified_session = measuring_session.verify_with_digest(
build,
sp_measurement,
&external_digest
)?;
```

2. PDH 직접 사용

```
// 인증서 체인 대신 PDH 직접 사용
let pdh_cert = get_platform_dh_certificate()?;
```

```
let start_data = session.start_pdh(pdh_cert)?;
```

3. 테스트용 모의 검증

```
// 테스트 환경에서만 사용
#[cfg(test)]
unsafe {
let mock_verified = session.mock_verify(measurement)?;
}
```

오류 처리 및 디버깅

1. 일반적인 세션 오류

```
use sev::error::SessionError;

match session_result {
Err(SessionError::CertificateError(e)) => {
eprintln!("인증서 오류: {:?}", e);
// 인증서 체인 확인 필요
},
Err(SessionError::CryptographicError(e)) => {
eprintln!("암호화 오류: {:?}", e);
```

```
// 키 생성 또는 암호화 과정 확인
},
Err(SessionError::RandomGenerationError(e)) => {
eprintln!("난수 생성 오류: {:?}", e);
// RDRAND 지원 확인
},
Ok(session) => {
println!("세션 생성 성공");
},
}
```

2. 측정 검증 실패

```
use std::io::ErrorKind;

match  verification_result {
Err(e) if  e.kind() == ErrorKind::InvalidInput => {
eprintln!("측정값 불일치 - 게스트 이미지 확인 필요");
// 디버깅 정보 출력
println!("예상 측정값: {:?}", expected_measurement);
println!("실제 측정값: {:?}", actual_measurement);
println!("정책: {:?}", policy);
println!("빌드 정보: {:?}", build);
```

```
},
Err(e) => {
eprintln!("검증 과정 오류: {:?}", e);
},
Ok(verified_session) => {
println!("측정 검증 성공");
},
}
```

3. 키 유도 디버깅

```
#[cfg(debug_assertions)]
fn debug_key_derivation(master: &Key, context: &[u8], label: &str) {
println!("=== 키 유도 디버그 ===");
println!("마스터 키: {}", hex::encode(master));
println!("컨텍스트: {}", hex::encode(context));
println!("레이블: {}", label);
let derived = master.derive(16, context, label).unwrap();
println!("유도된 키: {}", hex::encode(&derived));
}
```

보안 고려사항

1. 키 관리 보안

```
// 키는 항상 안전하게 생성되어야 합니다
let secure_key = Key::random(16)?; // RDRAND 사용

// 키는 사용 후 메모리에서 안전하게 제거됩니다
// Drop trait이 자동으로 처리

// 키를 복사할 때 주의
let key_copy = Key::new(secure_key.to_vec()); // 필요한 경우만
```

2. 타이밍 공격 방지

```
// 상수 시간 비교 사용
use subtle::ConstantTimeEq;

fn secure_compare(a: &[u8], b: &[u8]) -> bool {
if a.len() != b.len() {
return false;
}
a.ct_eq(b).into()
}
```

3. 세션 상태 검증

```
// 세션 상태를 항상 확인
fn ensure_verified_session(session: &Session<Verified>) {
// 컴파일 타임에 상태 보장
// Verified 상태에서만 비밀 주입 가능
}
```

성능 최적화

1. 키 캐싱

```
use std::collections::HashMap;

struct SessionCache {
derived_keys: HashMap<String, Key>,
}

impl SessionCache {
fn get_or_derive(&mut self, master: &Key, label: &str) -> Result<&Key> {
if !self.derived_keys.contains_key(label) {
let derived = master.derive(16, &[], label)?;
self.derived_keys.insert(label.to_string(), derived);
```

```
}
Ok(self.derived_keys.get(label).unwrap())
}
}
```

2. 배치 처리

```
// 여러 비밀을 한 번에 처리
fn  inject_multiple_secrets(
session: &Session<Verified>,
secrets: &[(HeaderFlags, &[u8])],
addresses: &[usize],
) -> Result<Vec<Secret>> {
secrets.iter()
.map(|(flags, data)| session.secret(*flags, data))
.collect()
}
```

모범 사례

1. 세션 생명주기 관리

```
struct SecureSession {
session: Option<Session<Verified>>,
```

```
}

impl SecureSession {
fn new(policy: Policy) -> Result<Self> {
let session = Session::try_from(policy)?;
Ok(Self { session: Some(session) })
}
fn inject_secret(&mut self, data: &[u8]) -> Result<Secret> {
let session = self.session.take()
.ok_or("세션이 이미 사용됨")?;
let secret = session.secret(HeaderFlags::default(), data)?;
// 세션은 한 번만 사용 후 소멸
Ok(secret)
}
}
```

2. 오류 복구

```
fn robust_session_creation(policy: Policy, max_retries: u32) -> Result<Session<
for attempt in 1..=max_retries {
match Session::try_from(policy) {
Ok(session) => return Ok(session),
Err(e) => {
```

```
eprintln!("세션 생성 실패 (시도 {}): {:?}", attempt, e);
if attempt == max_retries {
return Err(e.into());
}
std::thread::sleep(std::time::Duration::from_millis(100));
}
}
}
unreachable!()
}
```

3. 로깅 및 모니터링

```
use log::{info, warn, error};

fn monitored_session_verification(
session: Session<Measuring>,
build: Build,
measurement: Measurement,
) -> Result<Session<Verified>> {
info!("세션 측정 검증 시작");
let start_time = std::time::Instant::now();
```

```
let result = session.verify(build, measurement);

let duration = start_time.elapsed();

match &result {

Ok(_) => {

info!("세션 검증 성공 (소요시간: {:?})", duration);

},

Err(e) => {

error!("세션 검증 실패: {:?} (소요시간: {:?})", e, duration);

}

}

result

}
```

결론

SEV 세션 관리는 보안 가상화의 핵심 구성 요소로, 테넌트와 AMD 보안 프로세서 간의 신뢰할 수 있는 통신 채널을 제공합니다. 올바른 키 교환, 측정 검증, 비밀 주입 과정을 통해 게스트의 기밀성과 무결성을 보장할 수 있습니다. 특히 암호화 키의 안전한 관리와 상태 기반 세션 제어를 통해 높은 수준의 보안을 달성할 수 있습니다.